



**Università
degli Studi
di Palermo**



Array

CALCOLATORI ELETTRONICI – FONDAMENTI DI PROGRAMMAZIONE
a.a. 2025/2026

Prof. Roberto Pirrone



Array come struttura dati

- Un array è una struttura dati che contiene una sequenza di variabili tutte dello stesso tipo
- Sebbene il nome dell'array sia esso stesso una variabile, l'accesso ai singoli elementi della sequenza è assicurato specificando il *numero di posizione* di questi ultimi
- I numeri di posizione di un array di lunghezza $\mathcal{L}$ iniziano da 0 e vanno fino a $\mathcal{L} - 1$

Array come struttura dati

- Il tipo degli elementi può essere semplice o strutturato
- Abbiamo visto le stringhe di caratteri come array di `char`
- Vedremo array di `int`, `float`, `double`, ma anche array di array e array di altri tipi che ancora non abbiamo studiato

Dichiarazione di un array

```
SYNTAX:    element-type aname [size];           /* uninitialized */  
           element-type aname [size] = {initialization list}; /* initialized */
```

```
EXAMPLE:   #define A_SIZE 5  
           . . .  
           double a[A_SIZE];  
           char vowels[] = {'A', 'E', 'I', 'O', 'U'};
```

```
double cactus[5], needle, pins[6];
```

```
int     factor[12], n, index;
```

```
char vowels[] = "This is a long string";
```

*Posso mescolare dichiarazioni di array
e di variabili semplici*

Una stringa si inizializza con le « »



Università
degli Studi
di Palermo

dj
dipartimento
di ingegneria
unipa



Indicizzazione degli array

- Un elemento di un array può essere indicizzato attraverso una *qualsiasi espressione intera*

```
double x[8];
```

Array x

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]
16.0	12.0	6.0	8.0	2.5	12.0	14.0	-54.5



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Indicizzazione degli array

Cosa fanno queste istruzioni?

```
i = 5;
printf("%d %.1f\n", 4, x[4]);
printf("%d %.1f\n", i, x[i]);
printf("%.1f\n", x[i] + 1);
printf("%.1f\n", x[i] + i);
printf("%.1f\n", x[i + 1]);
printf("%.1f\n", x[i + i]);
printf("%.1f\n", x[2 * i]);
printf("%.1f\n", x[2 * i - 3]);
printf("%.1f\n", x[(int)x[4]]);
printf("%.1f\n", x[i++]);

printf("%.1f \n", x[--i]);
```

```
x[i - 1] = x[i];
x[i] = x[i + 1];
x[i] - 1 = x[i];
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Array e cicli

```
#define SIZE 11
```

```
int square[SIZE], i;
```

```
for (i = 0; i < SIZE; ++i)  
    square[i] = i * i;
```

Cosa fa questo codice?

Array square

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
0	1	4	9	16	25	36	49	64	81	100



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Array e cicli

- Scrivere un programma che calcola e stampa i quadrati dei primi k numeri con k richiesto all'utente
- Il programma deve:
 1. Definire un numero massimo di elementi NUM_MAX
 2. Richiedere all'utente il numero k di elementi e controllare che sia nel range [1, NUM_MAX]
 3. Calcolare i k quadrati
 4. Stamparli andando a capo tra un quadrato ed il successivo

Array e cicli

- Scriviamo un codice che richiede all'utente 10 elementi reali e ne calcola la media

$$\text{media} = \frac{1}{n} \sum_{i=0}^n x_i$$

Ricerca in un array

- Scriviamo un codice che ricerca un dato elemento all'interno di un array e mi dice se è presente o meno
- Suggerimenti:
 - Li guardiamo ad uno ad uno
 - Dobbiamo guardarli tutti?

Ordinare un array

- Scriviamo un codice che ordina un array di numeri, inizialmente disordinato, dal più piccolo al più grande
- Suggerimenti:
 - Mettiamo il più piccolo in prima posizione, poi il secondo più piccolo in seconda posizione e così via
 - Che vuol dire «Mettiamo»? Che fine fa l'elemento che già è in prima posizione, seconda posizione ... ?

Array e puntatori

- Eseguiamo il codice accanto
- Che significa questo risultato?
- Cos'è l'operatore * ?

```
#include <stdio.h>

int main(void){

    double x[] = {2.4, 5.6, 0.9, -12.4, 7.8};

    printf("x vale: %016lX\n\n",x);

    printf("*x vale: %lf\n", *x);

    return 0;
}
```

Usiamo il debugger!!!

Array e puntatori

- La variabile vettore di per sé è una variabile di tipo ***puntatore a double***
- Una variabile puntatore contiene *direttamente* il valore di un indirizzo di memoria
- In C un puntatore è *sempre* dichiarato in riferimento ad un tipo

Array e puntatori

- L'operatore `*` serve per accedere al contenuto del blocco di memoria riferita dalla variabile puntatore
- È il duale dell'operatore indirizzo `&`
- Si usa anche come modificatore di tipo per dichiarare esplicitamente una variabile puntatore a quel tipo:

```
double *p;
```

Array e puntatori

- Eseguiamo questo codice

- Come giustifichiamo il valore di p e di $*p$?

```
#include <stdio.h>
```

```
int main(){
```

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v;
```

```
printf("Il vettore v vale: %lX\n", v);  
printf("il puntatore p vale: %lX\n", p);  
printf("il contenuto di p vale: %lf\n", *p);
```

```
for(int i = 0; i < 5; i++){  
    printf("L'elemento v[%d] vale: %lf\n", i, v[i]);  
}
```

```
return 0;
```

```
}
```

E se volessi che p punti al terzo elemento di v ?

Array e puntatori

- Eseguiamo questo codice

- Come giustifichiamo il valore di p e di $*p$?

```
#include <stdio.h>
```

```
int main(){
```

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = &v[2];
```

```
printf("Il vettore v vale: %lX\n", v);  
printf("il puntatore p vale: %lX\n", p);  
printf("il contenuto di p vale: %lf\n", *p);
```

```
for(int i = 0; i < 5; i++){  
    printf("L'elemento v[%d] vale: %lf\n", i, v[i]);  
}
```

```
return 0;
```

```
}
```

E se volessi che p punti al terzo elemento di v ?



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Array e puntatori

```
#include <stdio.h>
```

```
int main(){
```

*E se volessi che p punti al
terzo elemento di v?*

Aritmetica dei puntatori ←

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v + 2;
```

```
printf("Il vettore v vale: %lX\n", v);  
printf("il puntatore p vale: %lX\n", p);  
printf("il contenuto di p vale: %lf\n", *p);
```

```
for(int i = 0; i < 5; i++){  
    printf("L'elemento v[%d] vale: %lf\n", i, v[i]);  
}
```

```
return 0;
```

```
}
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Array e puntatori

```
#include <stdio.h>
```

```
int main(){
```

*E se volessi che p punti al
terzo elemento di v?*

Aritmetica dei puntatori

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v + 2;
```

```
printf("Il vettore v vale: %lX\n", v);  
printf("il puntatore p vale: %lX\n", p);  
printf("il contenuto di p vale: %lf\n", *p);
```

*L'indirizzo di p è pari a
quello di v più*

due blocchi di dati double

```
for(int i = 0; i < 5; i++){  
    printf("L'elemento v[%d] vale: %lf\n", i, v[i]);  
}
```

```
return 0;
```

```
}
```

Array e puntatori

- Come faccio ad accedere ad un blocco dati puntato da un indirizzo calcolato in aritmetica dei puntatori?
- Uso l'operatore $*$

$$v[2] \rightarrow *(v + 2)$$

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v;
```

Quanto vale $(p + 3)$??*

Array e puntatori

- Come faccio ad accedere ad un blocco dati puntato da un indirizzo calcolato in aritmetica dei puntatori?
- Uso l'operatore $*$

$$v[2] \rightarrow *(v + 2)$$

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v;
```

Quanto vale $(p++)$??*

Array e puntatori

- Come faccio ad accedere ad un blocco dati puntato da un indirizzo calcolato in aritmetica dei puntatori?

```
double v[5] = {3.5, 7.6, 1.2, -3.2};  
double *p = v;
```

```
for(p = v; p < v + 5; p++){  
    printf("%lf\n", *p);  
}
```

Cosa fa questo codice ??

Argomenti dalla linea di comando

- Un modo molto diffuso di fornire dati ad un programma è quello di scriverli direttamente nella linea di comando subito dopo il nome del programma

```
prog opt1 opt2 opt3
```

- Questi dati vengono passati direttamente alla funzione `main` *esclusivamente in formato testo*, indipendentemente dal tipo desiderato



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Argomenti dalla linea di comando

- All'interno della funzione `main` la gestione è fatta così:

```
int
main(int  argc,      /* input - argument count (including
                      program name)
char *argv[]) /* input - argument vector
```

*/
*/

`char *``argv``[]`

Array di ... **puntatori a char** → Vettore di stringhe

Non è specificata la lunghezza né del vettore né delle singole stringhe

Argomenti dalla linea di comando

- All'interno della funzione `main` la gestione è fatta così:

```
int
main(int  argc,      /* input - argument count (including
                        program name)
                        */
      char *argv[]) /* input - argument vector
                        */
```

- E nel nostro esempio?

```
argc  4          argv[0]  "prog"
                        [1]  "opt1"
                        [2]  "opt2"
                        [3]  "opt3"
```